

Introduction to Information Retrieval

Feature Matching

Lecturer: Dr. LE Trung Nghia, Dr. NGUYEN Vinh Tiep

Disclaimer

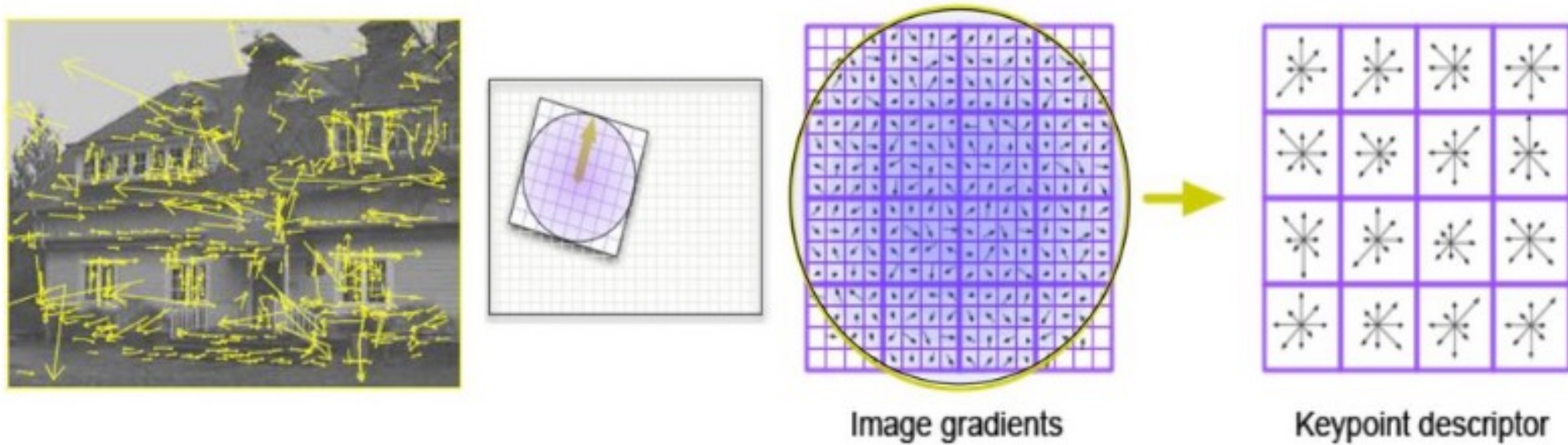
- Disclaimer: This curriculum have been inspired and adapted from materials created by:
 - Richard Szeliski, “Computer Vision: Algorithms and Applications”, 2nd edition, 2022. <https://szeliski.org/Book/>

Why Local Features?

- Motivation: Image matching
 - We have two images taken by two tourists
 - How do we ensure that these two pictures are from the same tourist spot?



SIFT: Scale Invariant Feature Transform



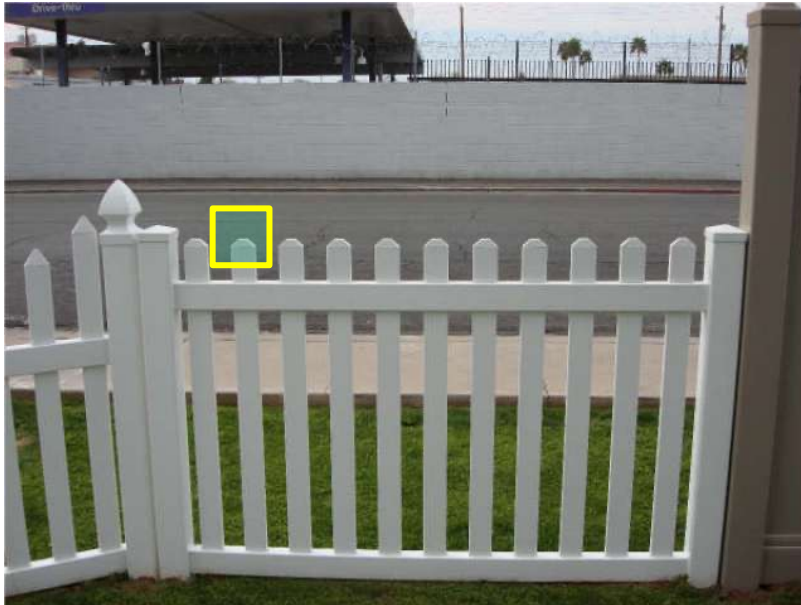
Feature Matching



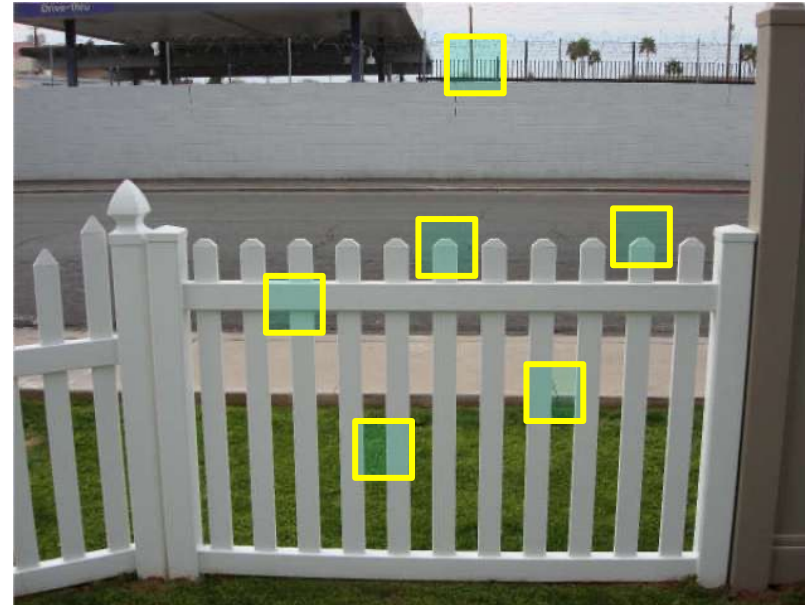
Feature Matching

Given a feature in I_1 , how to find the best match in I_2 ?

1. Define distance function that compares two descriptors
2. Test all the features in I_2 , find the one with min distance (exhaustive search)



I_1

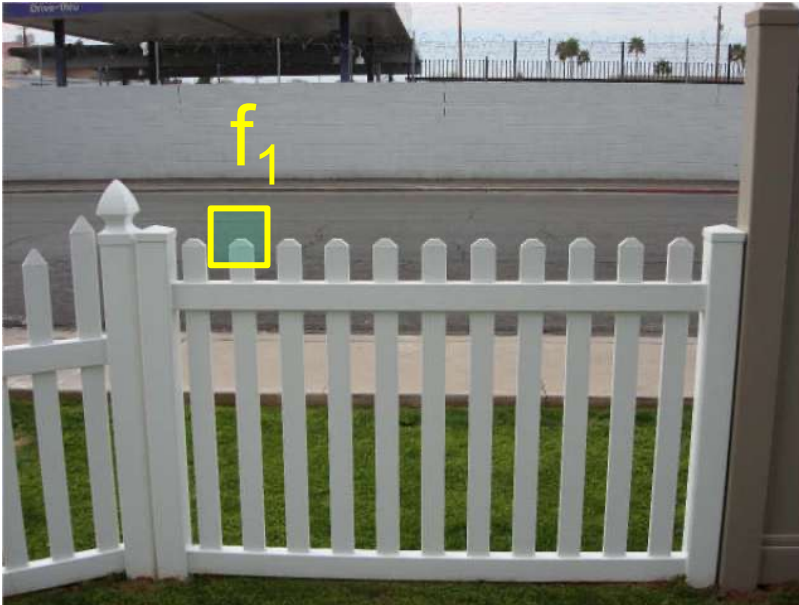


I_2

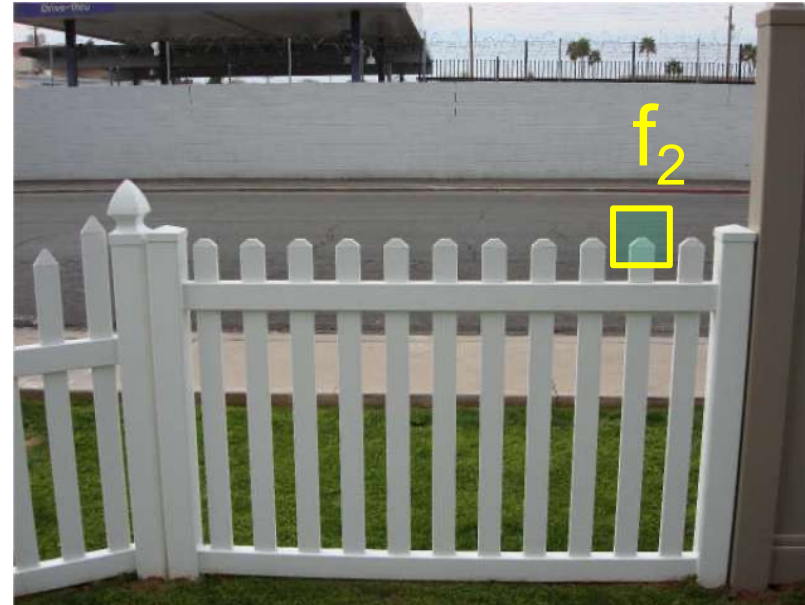
Feature Distance

How to define the difference between two features f_1, f_2 ?

- Simple approach is $SSD(f_1, f_2)$
 - Sum of square differences between entries of the two descriptors
 - Can give good scores to very ambiguous (bad) matches



I_1

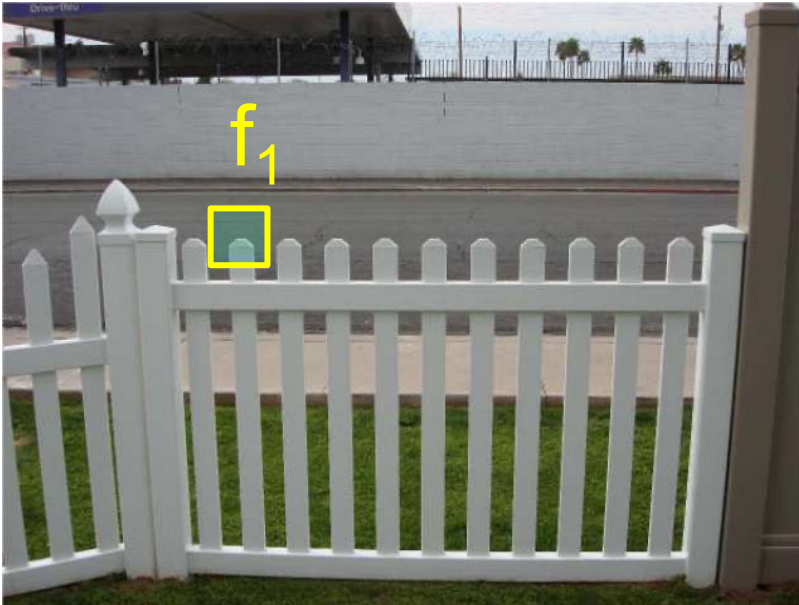


I_2

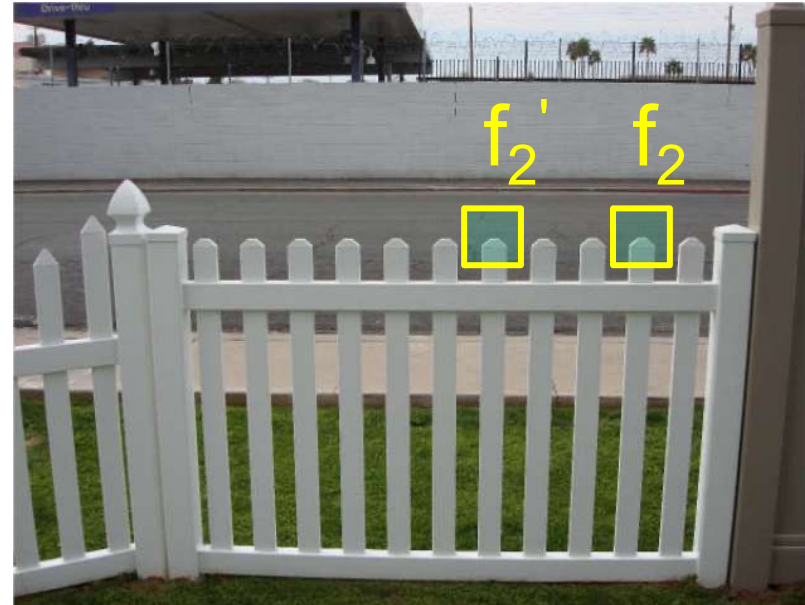
Feature Distance

How to define the difference between two features f_1, f_2 ?

- Better approach: ratio distance = $\text{SSD}(f_1, f_2) / \text{SSD}(f_1, f_2')$
 - f_2 is best SSD match to f_1 in I_2
 - f_2' is 2nd best SSD match to f_1 in I_2
 - Give small values for ambiguous matches



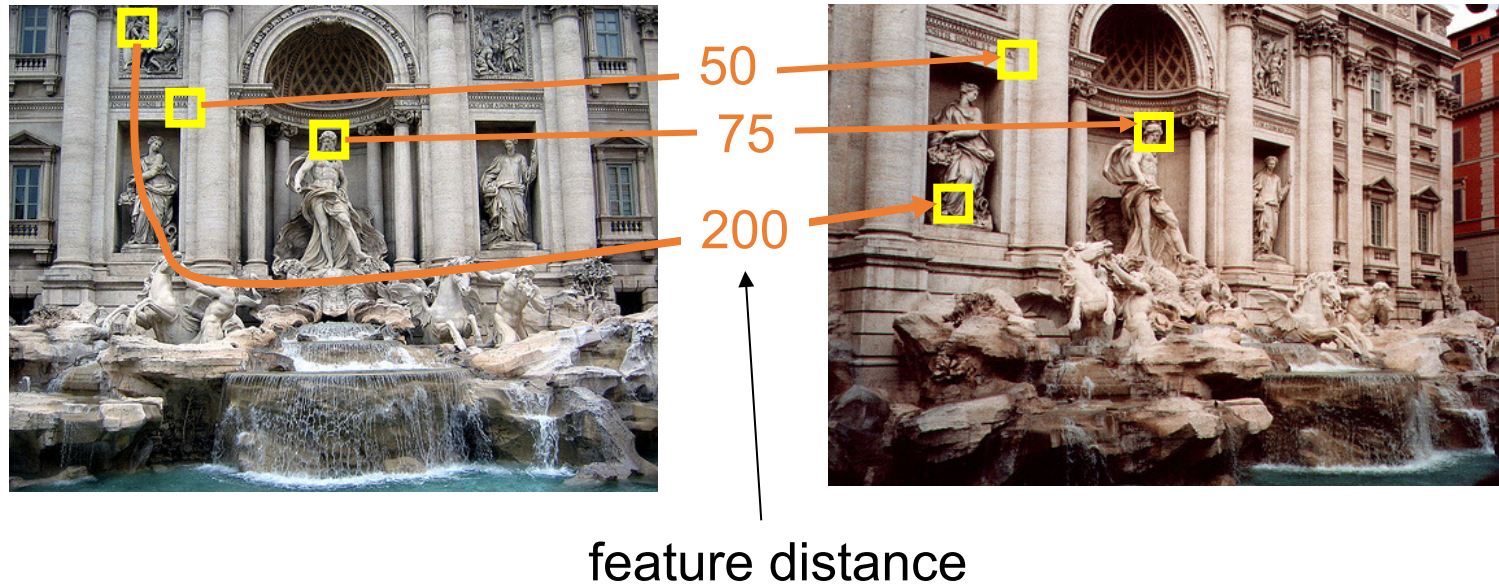
I_1



I_2

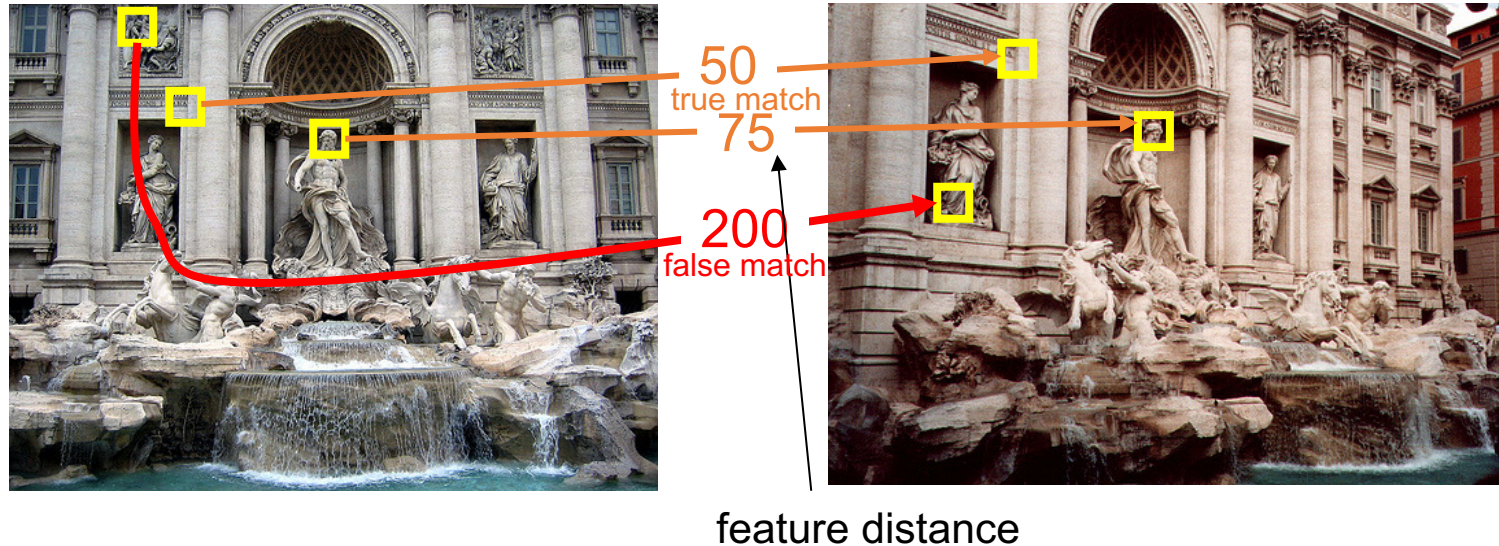
Evaluation

- How can we measure the performance of a feature matcher?



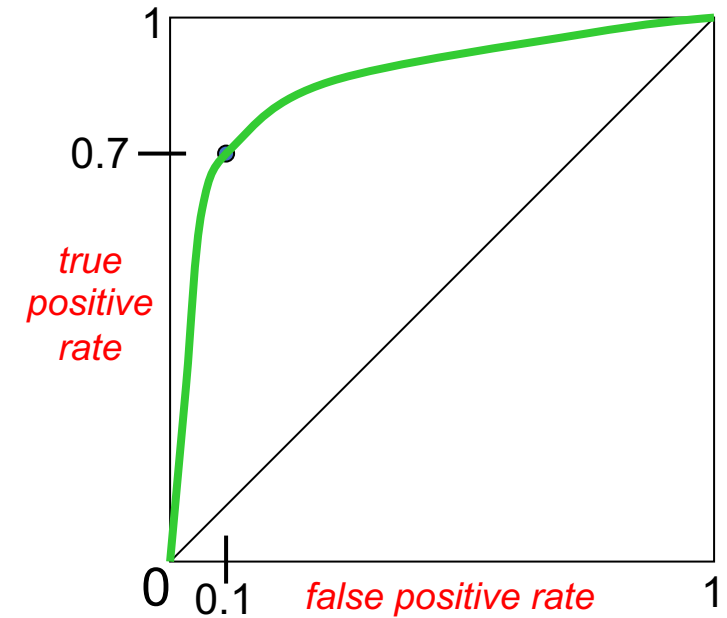
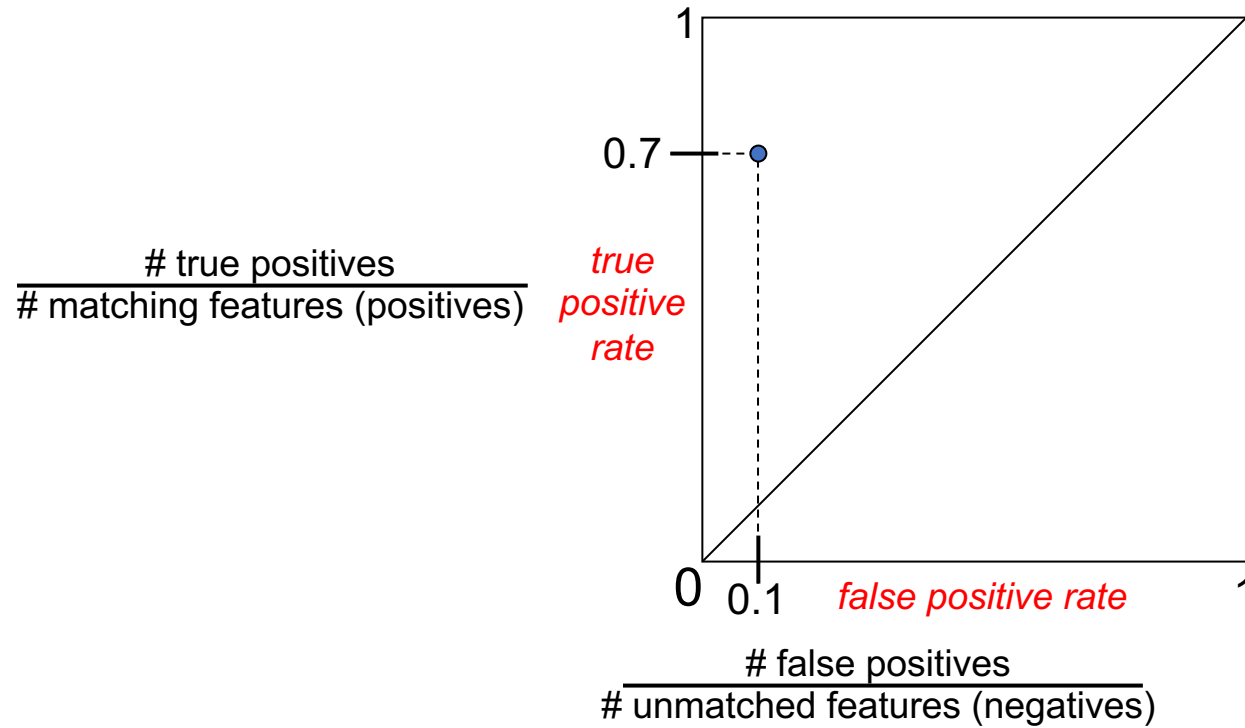
True-False Positive

- Issues of using a fixed threshold:



- The distance threshold affects performance
 - True positives = # of detected matches that are correct
 - Suppose we want to maximize these—how to choose threshold?
 - False positives = # of detected matches that are incorrect
 - Suppose we want to minimize these—how to choose threshold?

Evaluation



Possible Feature Matching Strategies

- Return all feature vectors with distance smaller than a threshold
- Nearest neighbor: feature vector with smallest distance
- Nearest neighbor distance ratio:
 - $NNDR = \frac{SSD(f_1, f_2)}{SSD(f_1, f'_2)}$
 - If NNDR is small, nearest neighbor is a good match

Feature Matching

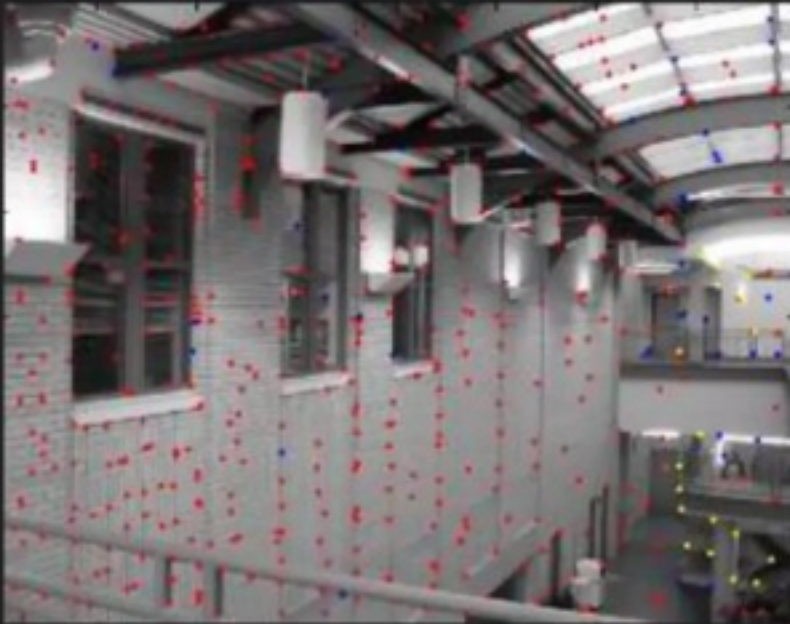
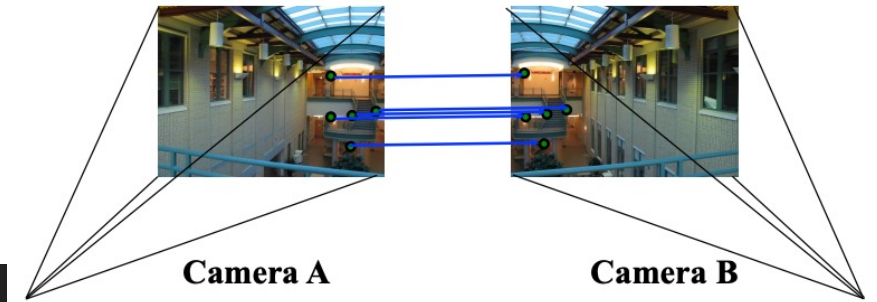
- Feature matching methods can give false matches
- Manually select good matches, or use robust method to remove false matches
- Nearest neighbor search is computationally expensive
 - Need efficient algorithm
 - Multi-dimensional search tree (K-D Trees & best bin first search). K-D Tree is not more efficient than exhaustive search for large dimensionality, e.g., >20
 - Hierarchical k-means tree
 - Approximate nearest neighbors: FLANN, FAISS

But...

- Remember: distinctive vs invariant competition? Means:

Problem: Even when pick best match, still lots (and lots) of wrong matches – “outliers”

Feature Matching Example



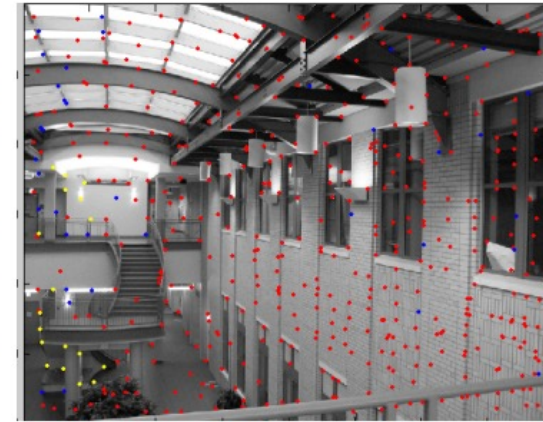
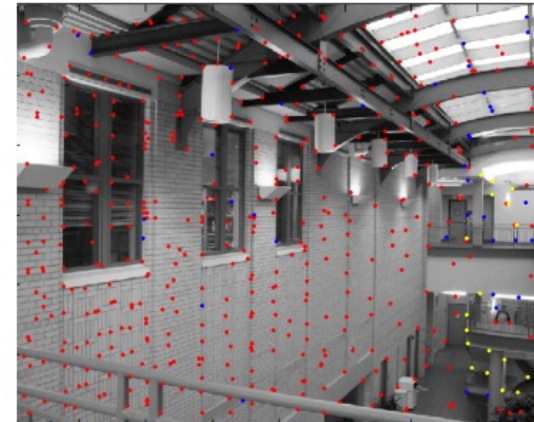
Red = bad match
match

Blue = good match

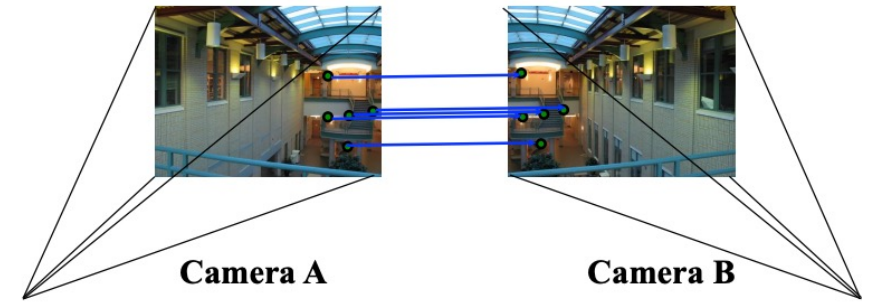
yellow = correct

Feature Matching Example

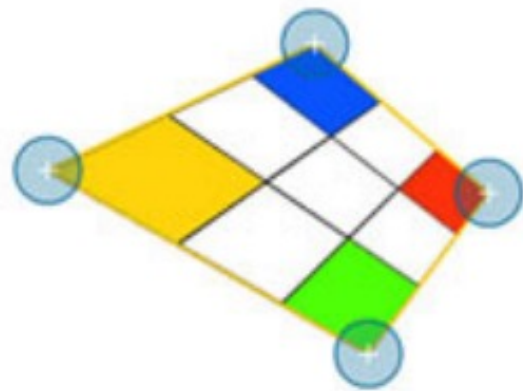
- **Red points:** points without a “good” match in the other image, measured by NNDR
- **Blue points:** These are points with a “good” match in which the match was wrong, meaning it connected two points that did not actually correspond in the world
- **Yellow points:** These are correct matches



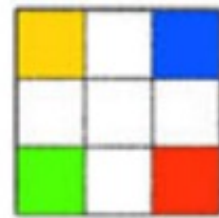
Homography Example



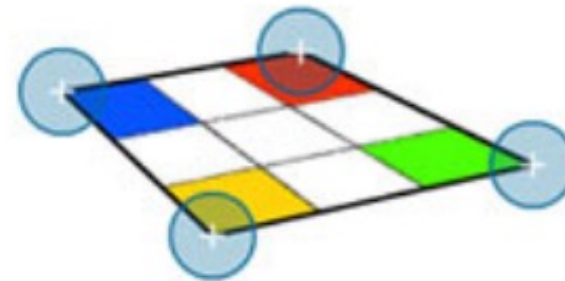
- A transformation (3x3 matrix) that maps the points in one image to the corresponding points in the other image



Original



homography



Distorted

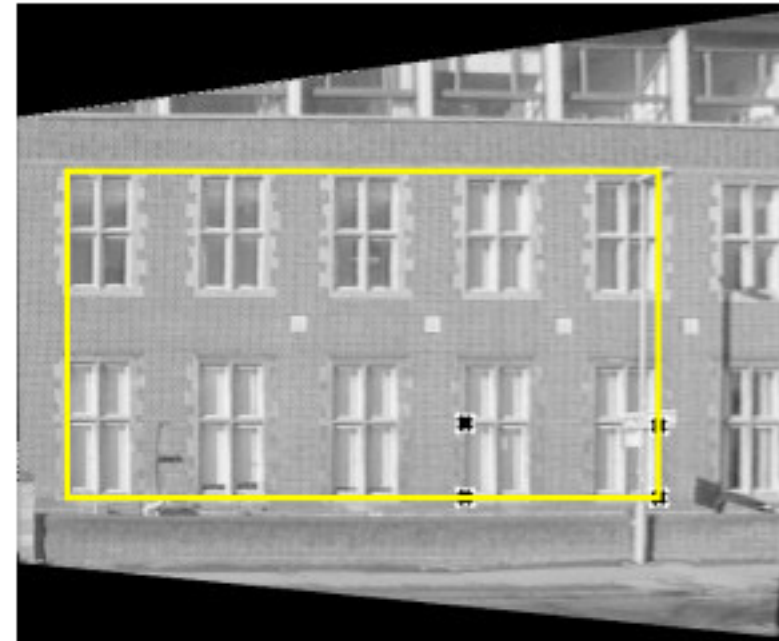
Direct Linear Transform (DLT)

- Formulation: $x' = Hx$, $x \propto Hx \rightarrow x'_i = Hx_i, i \in 1..n$
 $x'_i \times Hx_i = 0$
- 1. Get at least 4 point correspondences between two views
- 2. For each point correspondence, setup a partial A_i matrix of form 2×9
- 3. Assemble A matrix to form of 8×9 or $2n \times 9$
- 4. Apply Singular Value Decomposition: $A = UDV^T$
- 5. Assuming entries in D are sorted in descending order, the last column of V is solution to h . Reshape h 9×1 into H of form 3×3

$$\begin{bmatrix} -x_1 & -y_1 & -1 & 0 & 0 & 0 & x_1x'_1 & y_1x'_1 & x'_1 \\ 0 & 0 & 0 & -x_1 & -y_1 & -1 & x_1y'_1 & y_1y'_1 & y'_1 \\ -x_2 & -y_2 & -1 & 0 & 0 & 0 & x_2x'_2 & y_2x'_2 & x'_2 \\ 0 & 0 & 0 & -x_2 & -y_2 & -1 & x_2y'_2 & y_2y'_2 & y'_2 \\ -x_3 & -y_3 & -1 & 0 & 0 & 0 & x_3x'_3 & y_3x'_3 & x'_3 \\ 0 & 0 & 0 & -x_3 & -y_3 & -1 & x_3y'_3 & y_3y'_3 & y'_3 \\ -x_4 & -y_4 & -1 & 0 & 0 & 0 & x_4x'_4 & y_4x'_4 & x'_4 \\ 0 & 0 & 0 & -x_4 & -y_4 & -1 & x_4y'_4 & y_4y'_4 & y'_4 \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \\ h_6 \\ h_7 \\ h_8 \\ h_9 \end{bmatrix} = 0 \rightarrow h = \begin{pmatrix} h_1 \\ h_2 \\ \vdots \\ h_9 \end{pmatrix}, H = \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{bmatrix}$$

Homography Transformation

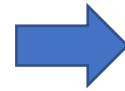
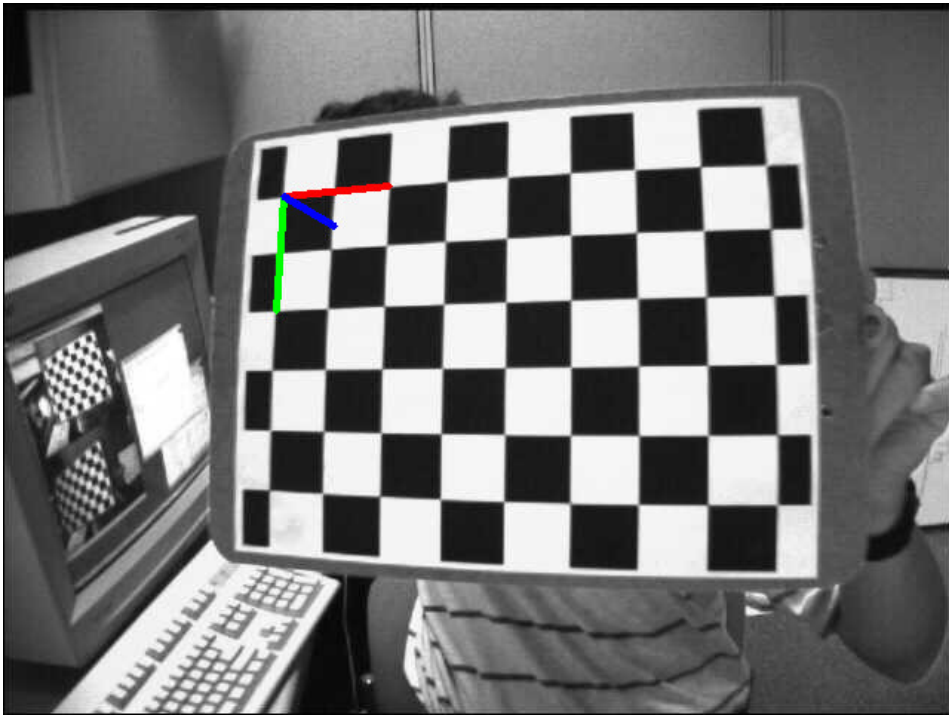
- Perspective removal / correction



from Hartley & Zisserman

Homography Transformation

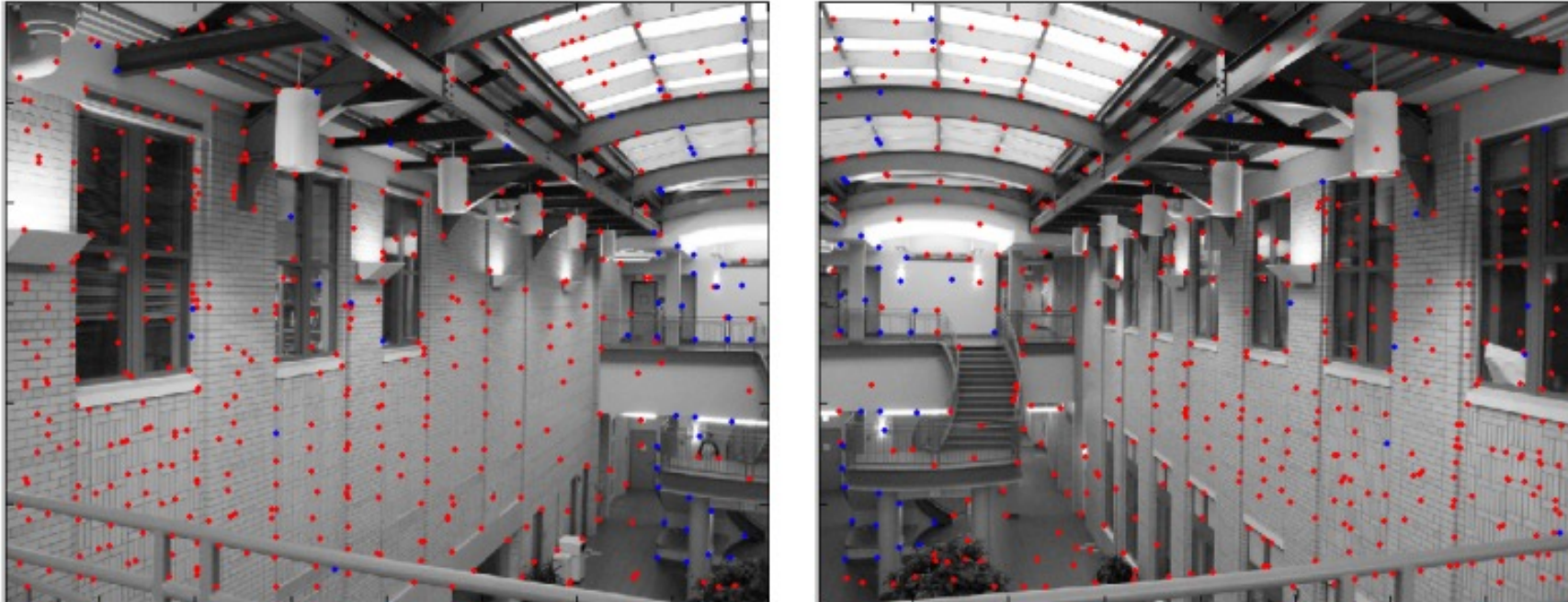
- Camera pose estimation from coplanar points for augmented reality with marker for instance



Example

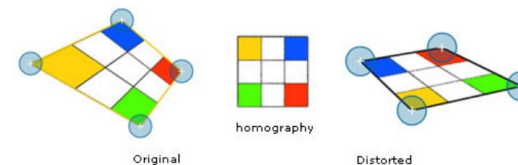


Feature-Space Outliner Rejection



Can we now compute H from the blue points?

- No! Still too many outliers...
- What can we do?



RANdom SAmple Consensus (RANSAC)

- Voting Scheme
- Idea: Maybe you do not need to have all samples have a vote.
 - Only a random subset of samples (points) vote

RANdom SAmple Consensus (RANSAC)

RANSAC

(**RAN**dom **SA**mples **C**onsensus) :

Fischler & Bolles in '81.



Algorithm:

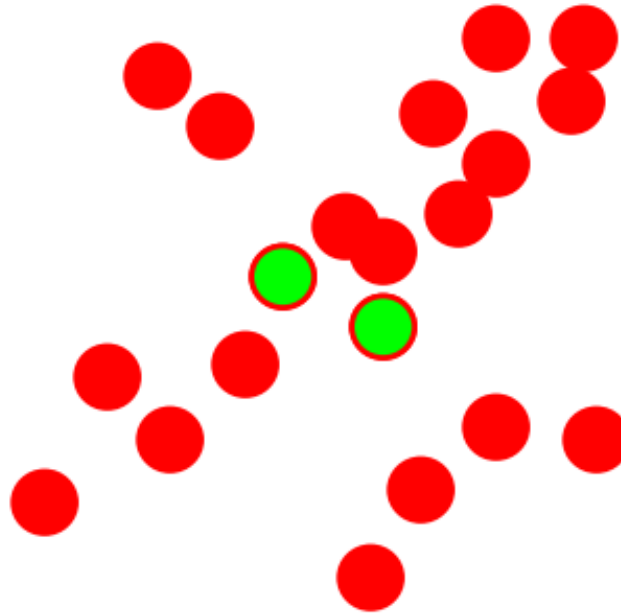
1. **Sample** (randomly) the number of points required to fit the model
2. **Solve** for model parameters using samples
3. **Score** by the fraction of inliers within a preset threshold of the model

Repeat 1-3 until the best model is found with high confidence

RANdom SAmple Consensus (RANSAC)

RANSAC

Line fitting example



Algorithm:

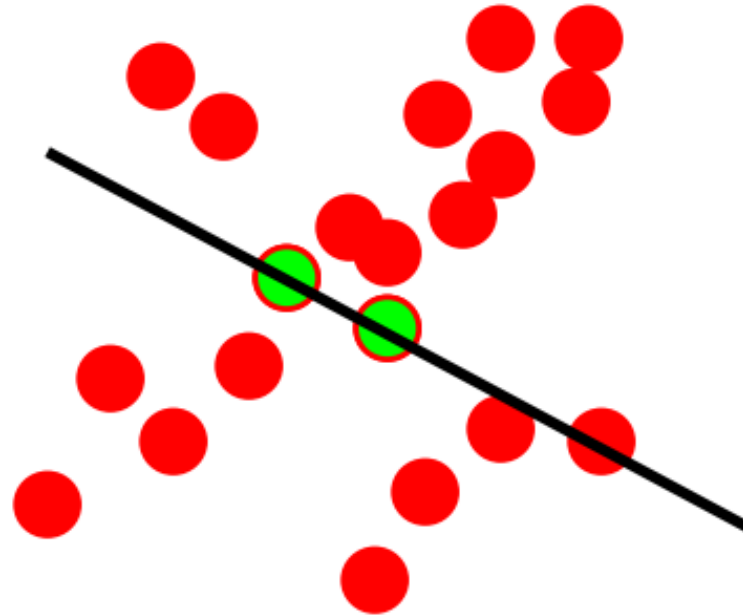
1. **Sample** (randomly) the number of points required to fit the model ($\# = 2$)
2. **Solve** for model parameters using samples
3. **Score** by the fraction of inliers within a preset threshold of the model

Repeat 1-3 until the best model is found with high confidence

RANdom SAmple Consensus (RANSAC)

RANSAC

Line fitting example



Algorithm:

1. **Sample** (randomly) the number of points required to fit the model ($\# = 2$)
2. **Solve** for model parameters using samples
3. **Score** by the fraction of inliers within a preset threshold of the model

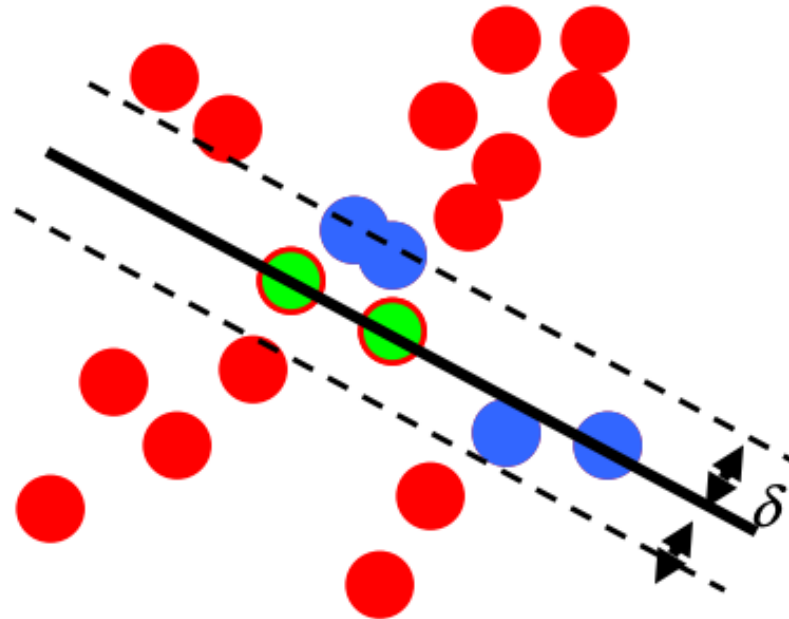
Repeat 1-3 until the best model is found with high confidence

RANdom SAmple Consensus (RANSAC)

RANSAC

Line fitting example

$$N_I = 6$$



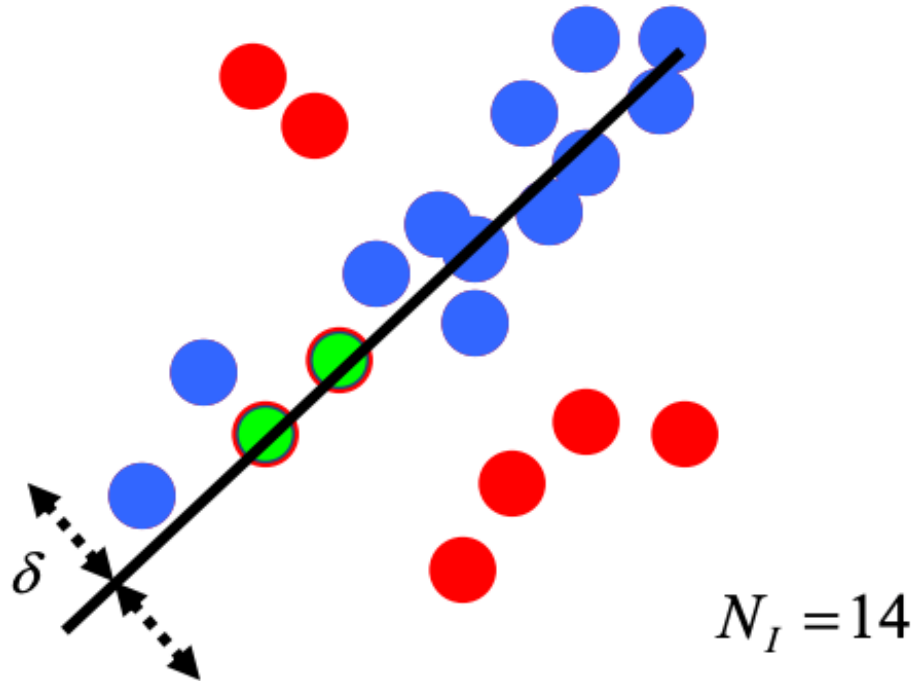
Algorithm:

1. **Sample** (randomly) the number of points required to fit the model ($\#=2$)
2. **Solve** for model parameters using samples
3. **Score** by the fraction of inliers within a preset threshold of the model

Repeat 1-3 until the best model is found with high confidence

RANdom SAmple Consensus (RANSAC)

RANSAC

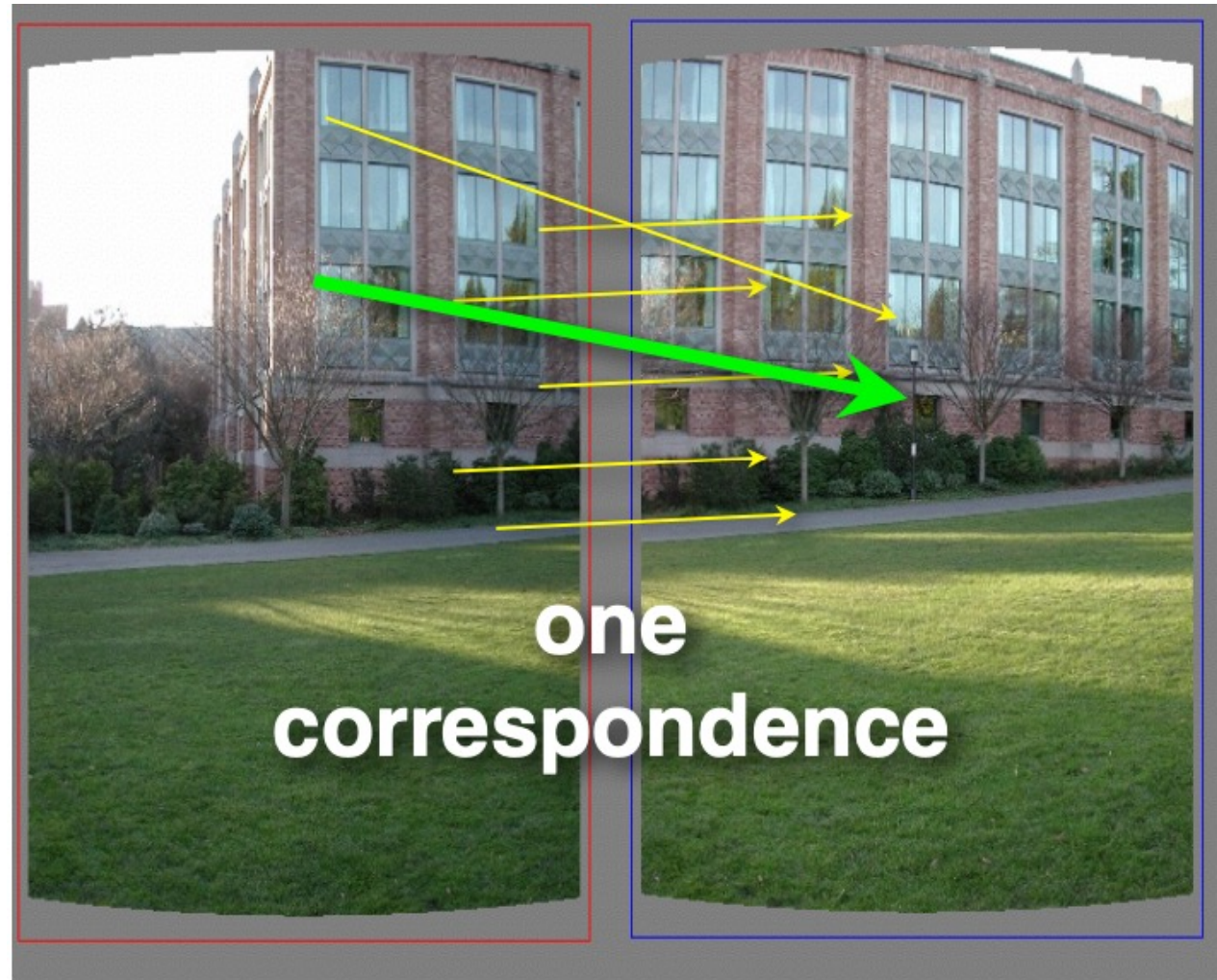


Algorithm:

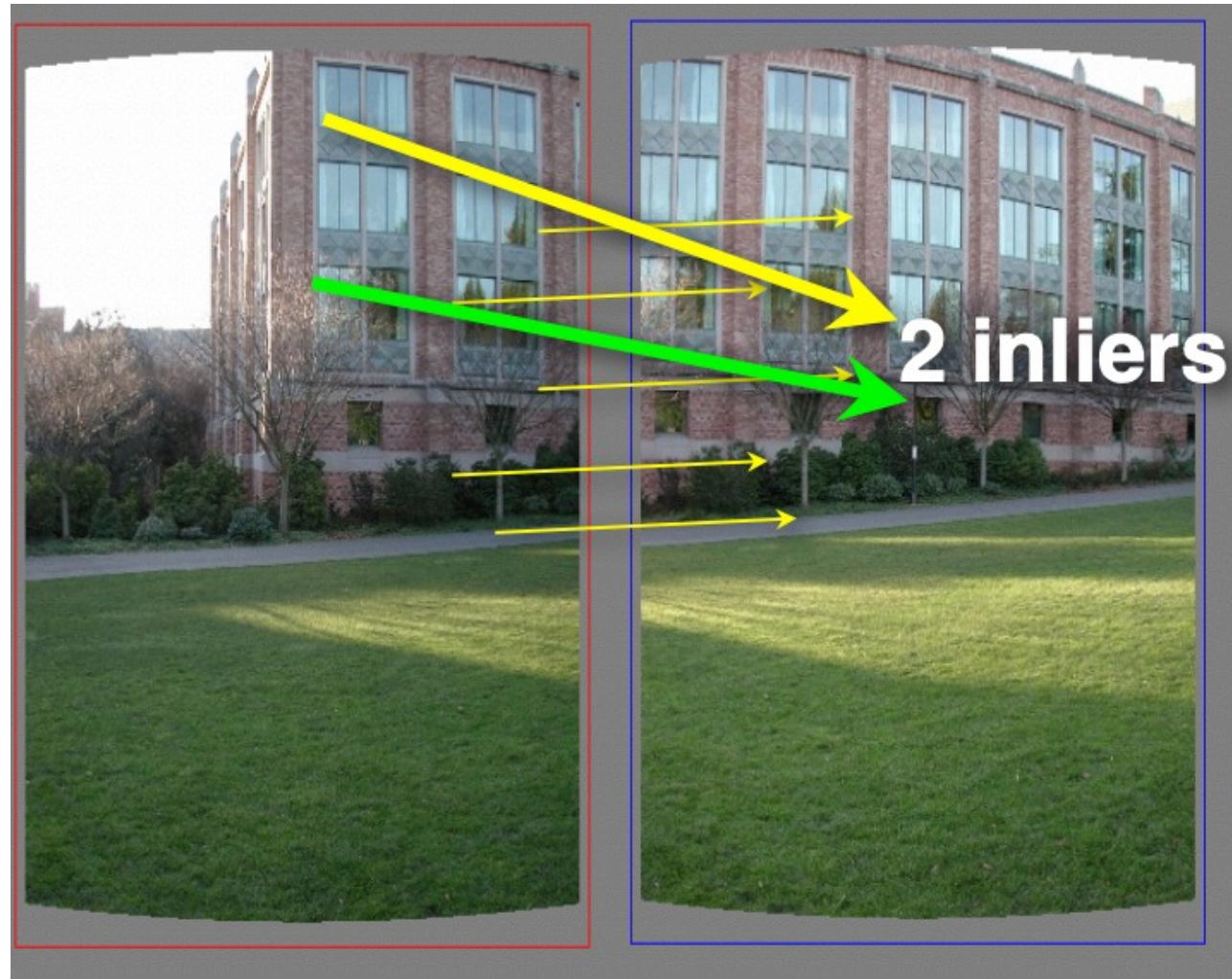
1. **Sample** (randomly) the number of points required to fit the model ($\# = 2$)
2. **Solve** for model parameters using samples
3. **Score** by the fraction of inliers within a preset threshold of the model

Repeat 1-3 until the best model is found with high confidence

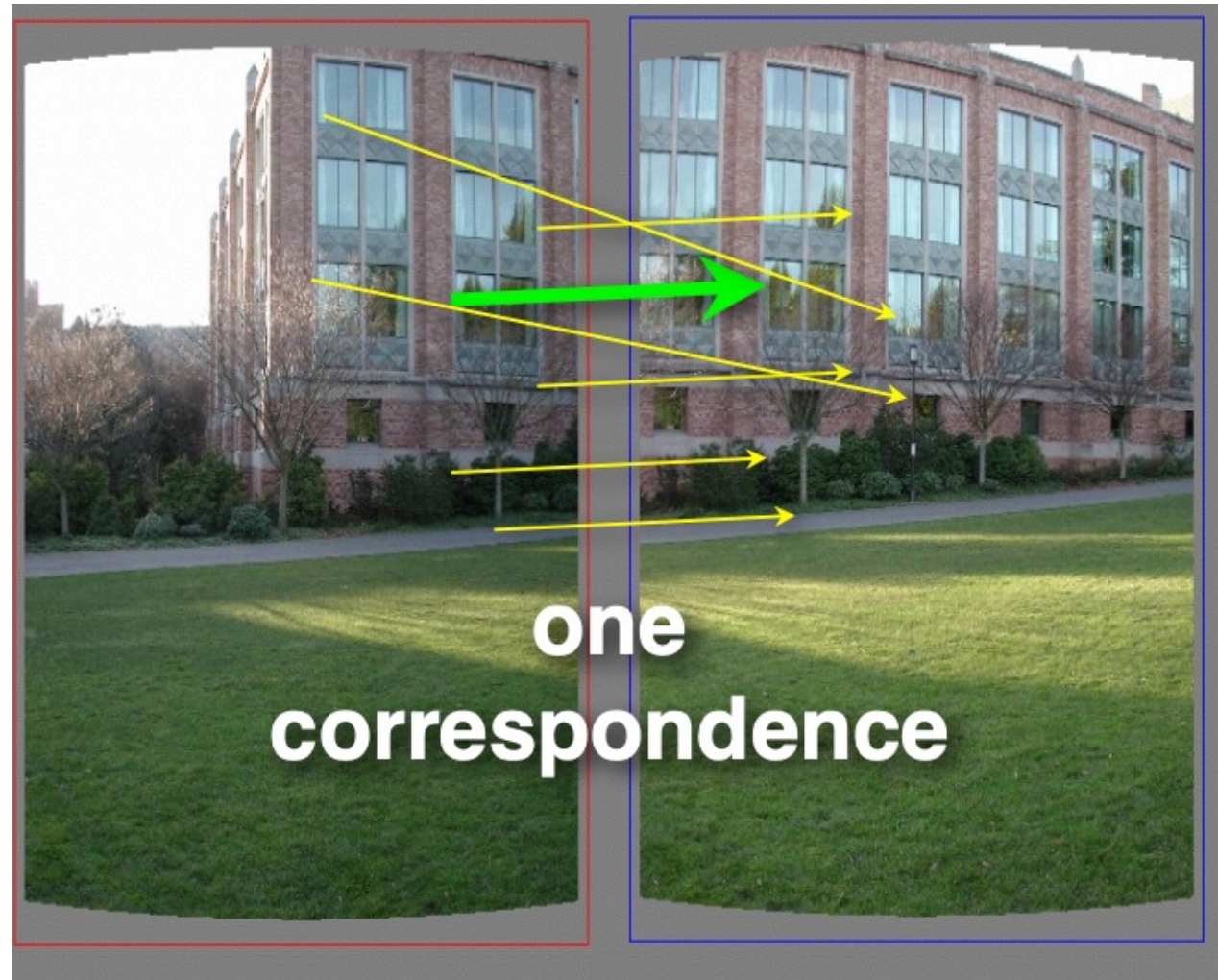
RANdom SAmple Consensus (RANSAC)



RANdom SAmple Consensus (RANSAC)



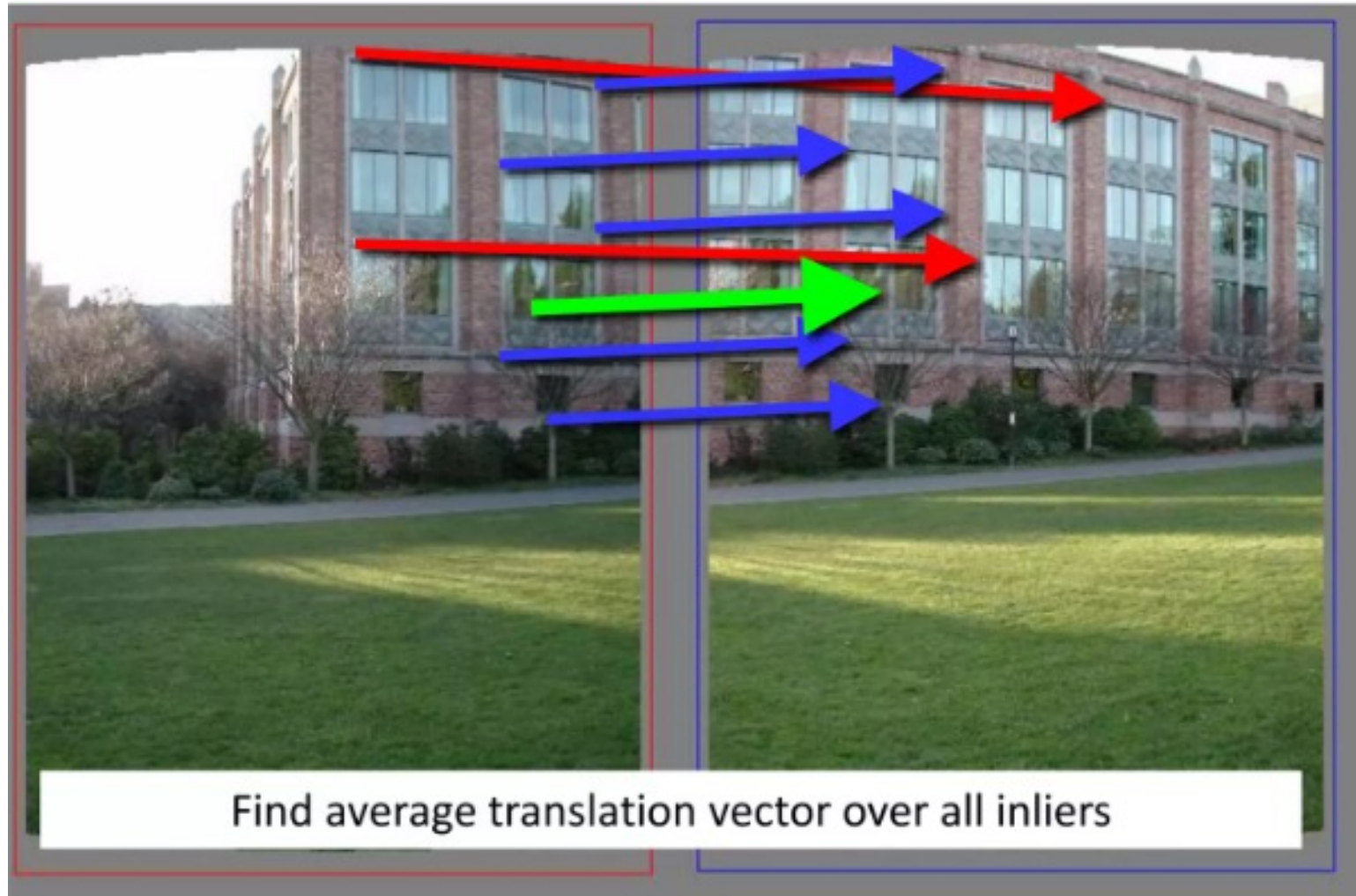
RANdom SAmple Consensus (RANSAC)



RANdom SAmple Consensus (RANSAC)



RANdom SAmple Consensus (RANSAC)



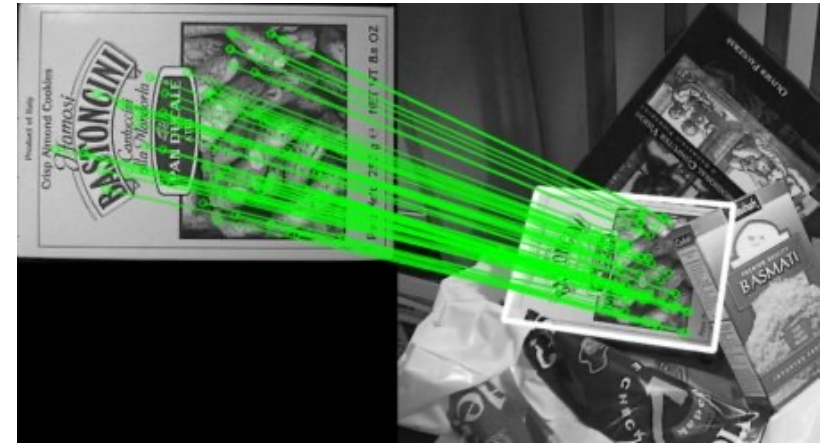
RANdom SAmple Consensus (RANSAC)

- RANSAC for estimating homography matrix
- RANSAC loop:
 - 1. Select four feature pairs (at random)
 - 2. Compute homography H (exactly) using Direct Linear Transform (DLT)
 - 3. Compute inliers where $SSD(p'_i, Hp_i) < \epsilon$
 - 4. Keep H if largest number of inliers
- Re-compute least-squares H estimate on all of the inliers

RANSAC in OpenCV

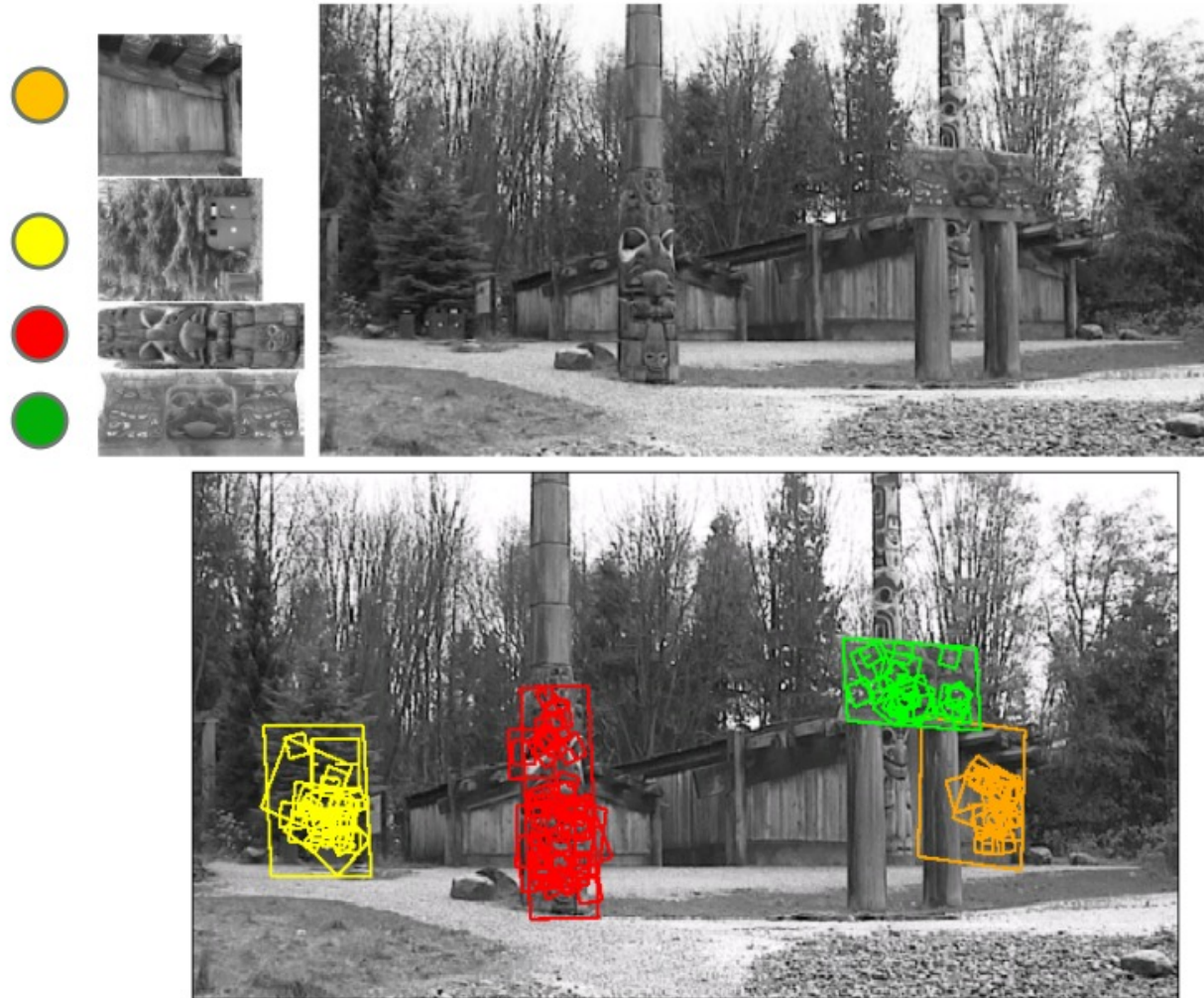
- `sift = cv2.SIFT_create()`
- `kp1, des1 = sift.detectAndCompute(img1, None)`
- `kp2, des2 = sift.detectAndCompute(img2, None)`
- `FLANN_INDEX_KDTREE = 0`
- `index_params = dict(algorithm = FLANN_INDEX_KDTREE, trees = 5)`
- `search_params = dict(checks = 50)`
- `flann = cv2.FlannBasedMatcher(index_params, search_params)`
- `matches = flann.knnMatch(des1, des2, k=2)`
- `good = []`
 - `for m, n in matches:`
 - `if m.distance < 0.7*n.distance:`
 - `good.append(m)`

RANSAC in OpenCV



- MIN_MATCH_COUNT = 10
- if `len(good) > MIN_MATCH_COUNT`:
 - `src_pts = np.float32([ kp1[m.queryIdx].pt for m in good ]).reshape(-1,1,2)`
 - `dst_pts = np.float32([ kp2[m.trainIdx].pt for m in good ]).reshape(-1,1,2)`
 - `M, mask = cv2.findHomography(src_pts, dst_pts, cv.RANSAC, 5.0)`
 - `matchesMask = mask.ravel().tolist()`
 - `h,w = img1.shape`
 - `pts = np.float32([ [0,0],[0,h-1],[w-1,h-1],[w-1,0] ]).reshape(-1,1,2)`
 - `dst = cv2.perspectiveTransform(pts, M)`
 - `img2 = cv2.polylines(img2, [np.int32(dst)], True, 255, 3, cv.LINE_AA)`
- `draw_params = dict(matchColor = (0,255,0), singlePointColor = None, matchesMask = matchesMask, flags = 2)`
- `img3 = cv2.drawMatches(img1, kp1, img2, kp2, good, None, **draw_params)`

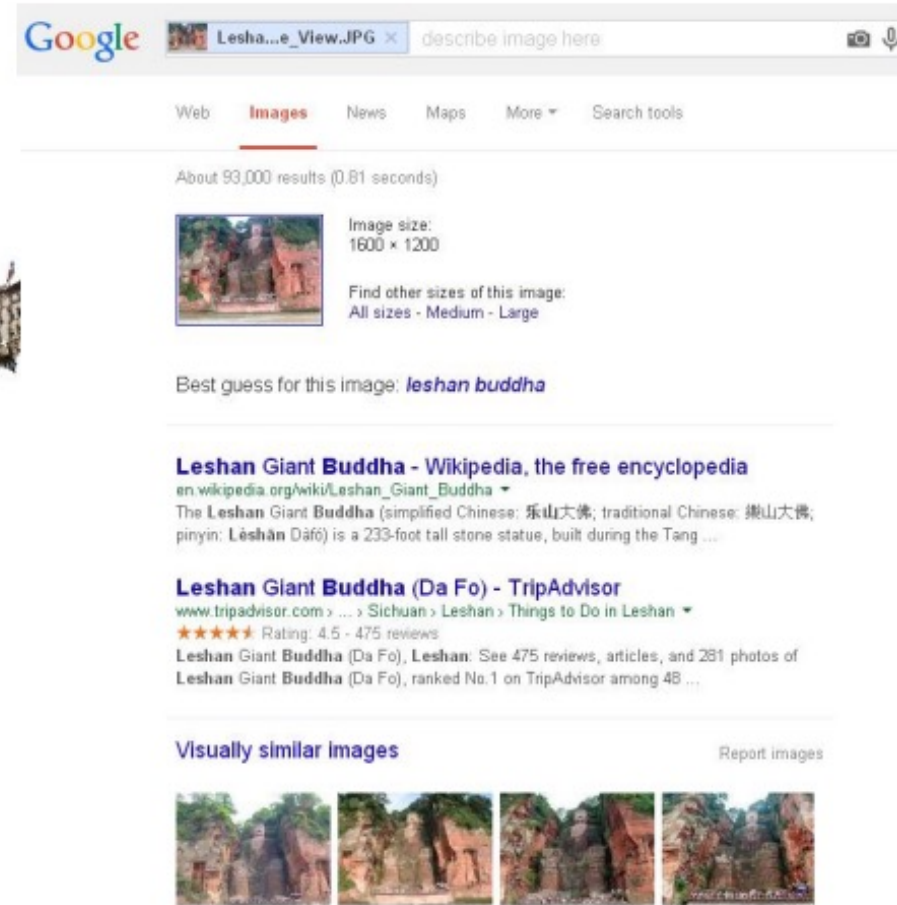
SIFT Matching Object Pieces



Applications



Dense city 3D reconstruction/ Structure from motion



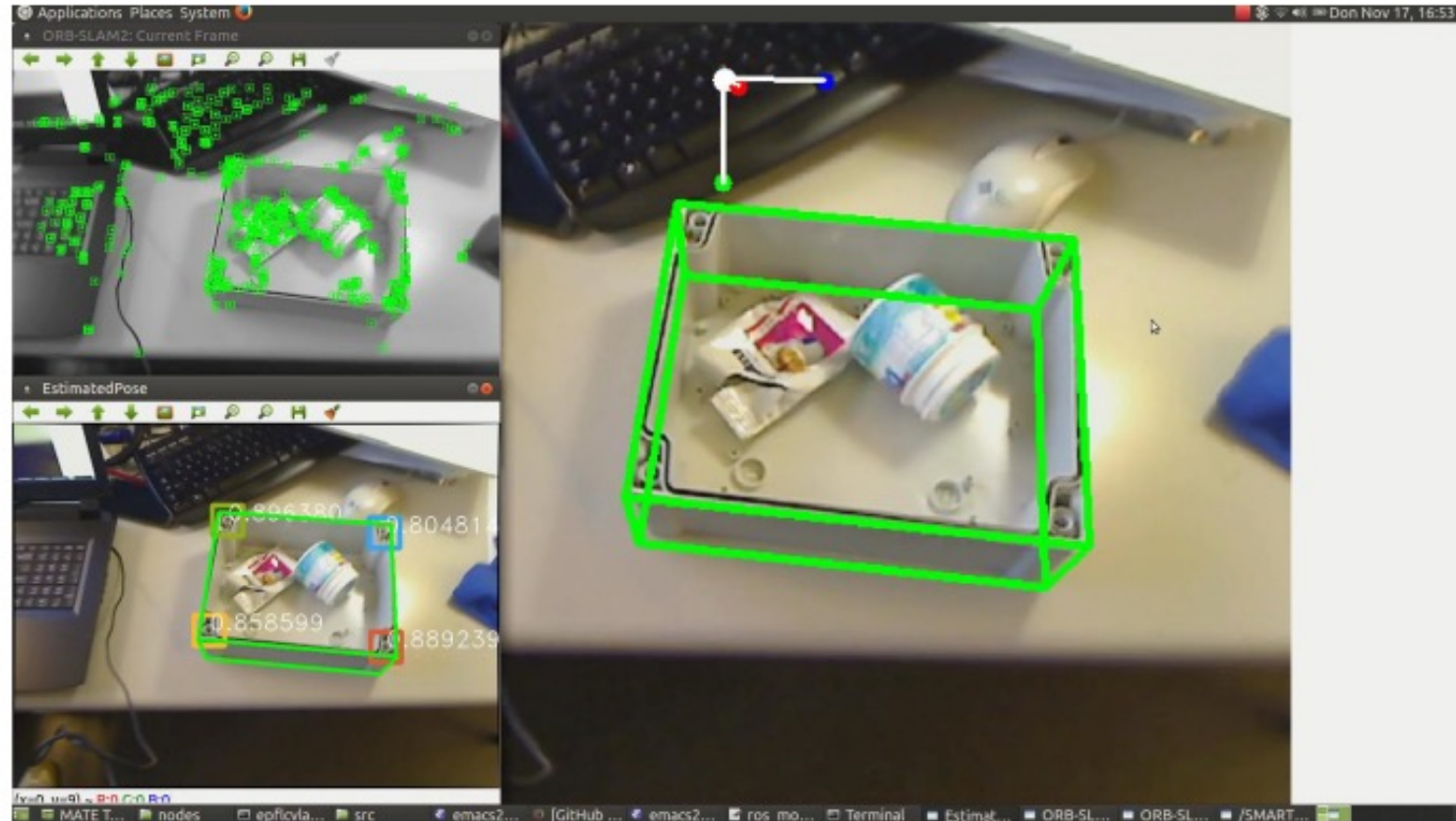
Content-based web image search

Applications



... detect objects in crowded scenes.

Applications



... track objects in real-time even when there are occlusions and motion blur.

Applications

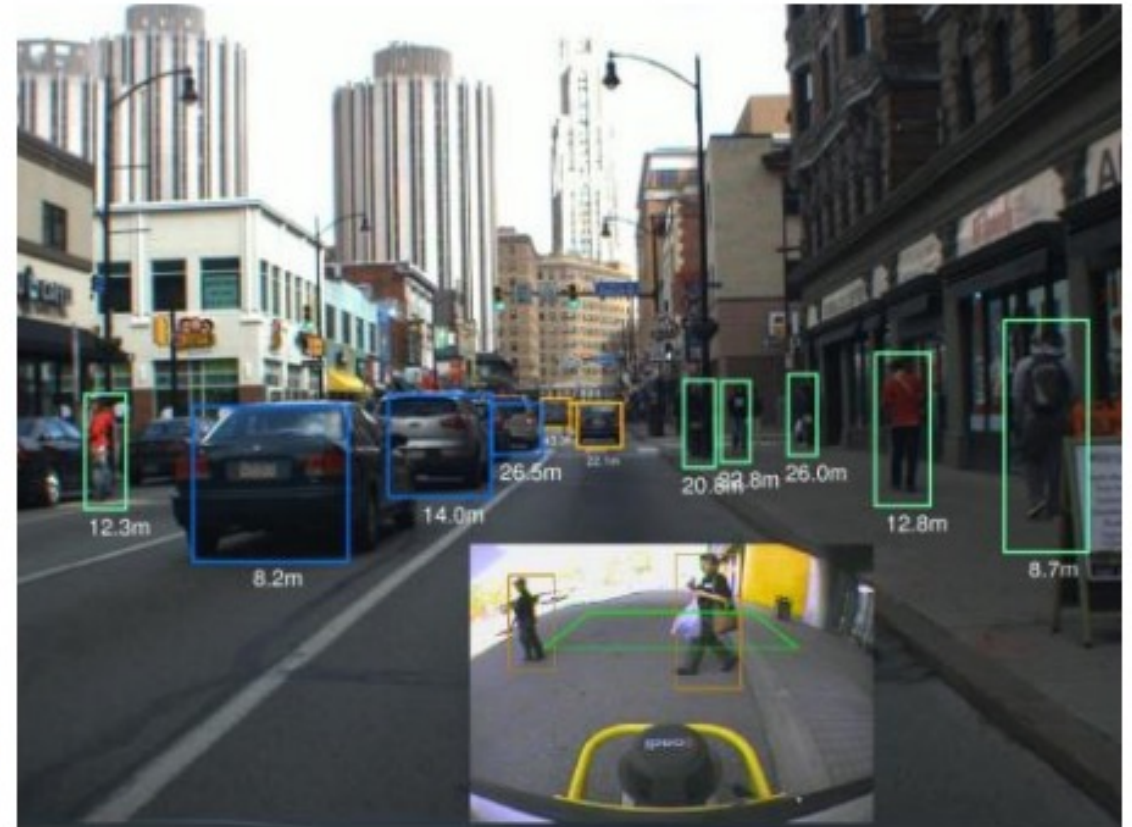


... mosaic images into panoramas.

Applications



Mobile augmented reality



Real-time pedestrian detection